

PanTS Review

Finding pancreatic tumors in 3D CT — a segmentation and annotation-assist tool

Quinn Evans · five-week independent ML project · Johns Hopkins PanTS dataset

Research use only — this is not a diagnostic system.

Why the pancreas is hard

- Pancreatic cancer is among the deadliest — outcomes hinge on catching it early
- The organ is soft, irregular, and hidden behind other structures
- Tumors are often nearly invisible to the human eye on a CT slice
- A radiologist reads hundreds of slices per scan, and most scans are normal

10.4%

of scans in this dataset
contain a tumor

A needle-in-a-haystack read, repeated all day.

Who it's for, and what changes

Today

- Trace the pancreas by hand, slice by slice
- Hunt for a lesion that may not be visible
- Slow, tedious, and easy to miss something

With this tool

- A proposed outline is already drawn
- Accept, edit, or reject it
- A flag says “look here first”
- Export the mask and finish the job

User: radiologist / imaging annotator

What this is — and what it isn't

It IS

A segmentation tool: it outlines the pancreas and any suspicious region

It IS

A CADe assist: it flags “there could be a tumor here” for review

It is NOT

A diagnosis — it never decides whether a patient has cancer

It is NOT

Autonomous — a human reviews and edits every single output

Three things to watch

1

An unmarked scan

Exactly what a radiologist sees —
nothing drawn on it yet

2

The model scores it live

A real call to the deployed endpoint,
about half a second

3

The truth is revealed

Expert contours overlaid on the
prediction — how close was it?

Every scan shown is held-out data the model never trained on.

LIVE DEMO

PanTS Review · localhost:5173 · live inference on port 8000

What just happened

1

Real model

The registered model, version 1 — the same checkpoint I evaluated

2

Real call

A live HTTP request to a deployed FastAPI endpoint, ~0.6s per scan

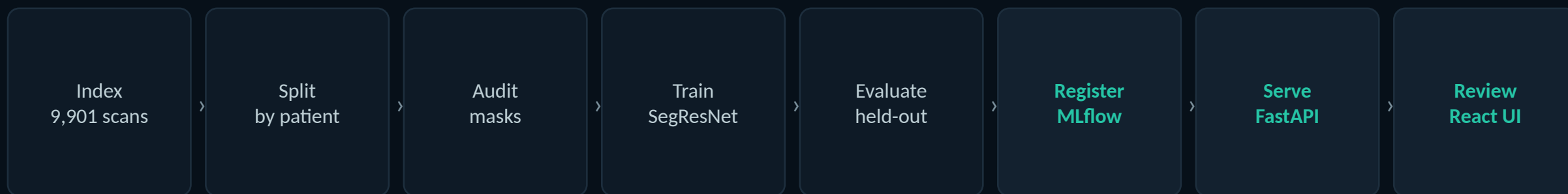
3

Real data

Held-out scans from the official test split — never seen in training

Not a recording, not a screenshot.

End to end



Config-driven

One YAML defines the recipe; nothing hardcoded

Reproducible

Fixed seed, committed splits, every run logged to MLflow

Guarded

Startup abort if a training split ever touches validation

The data was the hard part

- 9,901 real CT volumes — about 380 GB, on an external drive, never in the repo
- Split by patient, stratified by tumor, so no patient appears on both sides
- Roughly 1 in 8 scans had a broken or empty mask — audited and excluded
- Volumes range from 8 to over 1,000 slices, across four scanner manufacturers

9,901

CT volumes indexed

~1 in 8

masks broken or empty

Auditing data integrity turned out to be the work, not a distraction from it.

The model

3D SegResNet

- Fine-tuned from SuPreM — pretrained by the same Johns Hopkins lab
- Three classes: background, pancreas, lesion
- Whole-box input: the entire pancreas region as one cube
- Trained on Apple Silicon (MPS), ~7-14 h per run

Does the pretraining matter?

I measured it instead of assuming.

0.120

from scratch

0.257

SuPreM transfer

+0.13 lesion Dice — the pretraining is what makes this work at my data scale.

From checkpoint to product

MLflow registry

- pancreas-lesion-segmenter v1
- Checkpoint step 18000
- SHA + config + git commit logged

FastAPI endpoint

- GET /health, GET /cases
- POST /predict → CADe summary
- ~0.6 s per scan

React + NiiVue UI

- Live analyze against the endpoint
- Falls back to cached results
- 3D surfaces from marching cubes

The model is versioned, served, and consumed — not a notebook artifact.

26 experiments, bars set in advance

- Every experiment: a hypothesis and an accept/reject bar written BEFORE the run
- One variable at a time — everything else held fixed
- Each result logged with its decision, including the failures
- Roughly one experiment per night: a training run is 7 to 14 hours

26

numbered experiments

Accepted, rejected, or null —
all of them written down.

If I couldn't say in advance what would make me abandon an idea, I wasn't testing it.

The breakthrough was an idea, not a knob

- The model found tumors but over-predicted them — flagging something on 92% of healthy scans
- I tried the obvious knobs. Sampling: null. Loss function: null.
- Two nulls told me the problem was structural, not a hyperparameter
- So: stop feeding random patches. Feed the ENTIRE pancreas region as one cube.

One change

Pancreas Dice **0.72 → 0.81**

Specificity **8% → 55%**

Lesion Dice **new best**

Became the architecture of the final model.

I found out my own number was wrong

- An adversarial code audit found my data-scaling script sampled from the wrong column
- My training set overlapped my validation set — 30 of the 40 cases I was reporting on had been trained on
- I verified it myself, then fixed the root cause rather than the symptom
- Added a guard that aborts training if a split ever touches validation — that class of bug can't recur silently
- Rebuilt the splits, retrained, and re-reported the honest number

Reported

~~0.528~~

Honest

0.474

Lower, real — and it didn't collapse, so the underlying result was genuine.

Two experiments I rejected

Anatomy-aware supervision

Looked like a clear win at an intermediate checkpoint — +0.041 lesion Dice.

I trained both arms to convergence before deciding. The effect vanished. Rejected.

More healthy training data

Cleared its specificity bar decisively — 17% up to 46%.

But detection fell to 88%, three cases below the floor I set in advance. Rejected.

A result you stop at because it looks good is not a result you can trust.

RESULTS

Final numbers

Official held-out test set — 901 scans, scored once, never touched during development

96%

detection sensitivity
145 of 151 tumors flagged

0.474

lesion Dice
95% CI 0.42 – 0.52

0.827

pancreas Dice

17%

specificity
on healthy scans

Published PanTS benchmark: ~0.53 lesion Dice (MedFormer 0.529 · R-Super 0.534, which uses additional external data)

What those numbers mean for the user

96% detection

Catch rate. Of the tumors present, the tool surfaces almost all of them for review.

0.474 lesion Dice

Edit burden. The outline is a usable starting point that still needs a human pass.

0.827 pancreas Dice

The organ outline is close to expert quality — the tedious tracing is largely done.

17% specificity

False-alarm rate. It cries wolf often, so every flag is a prompt, never a verdict.

Limitation: the model is given the pancreas region

It is pointed at the pancreas rather than searching the whole scan on its own.

Why that was the right call for this project

- It is a real workflow — in annotation-assist, the reviewer supplies the region
- It isolates segmentation quality from localization error, so the number measures one thing
- That constraint IS the whole-box breakthrough — the fix for over-prediction came from it
- Not ignored: I built and evaluated the autonomous version, with a millimetre-level containment audit

Limitation: specificity is low

It over-calls on healthy scans — 17% specificity is the weakest number in the project.

Deliberately deprioritised — and worked, not forgotten

- For a CADe tool, a missed tumor costs more than a false alarm — detection came first
- Four documented attempts: sampling ratio, loss function, an anatomical constraint, more healthy data
- The model can discriminate — patient-level AUC 0.80 — so it is a threshold problem, not blindness
- The honest conclusion: it needs a decision layer in front of it, not a more timid segmenter

Where it fails, precisely

Small $< 1 \text{ cm}^3$

0.067

lesion Dice · 78% detected

Medium $1-8 \text{ cm}^3$

0.512

lesion Dice · 100% detected

Large $> 8 \text{ cm}^3$

0.610

lesion Dice · 100% detected

Small tumors are found but over-drawn — sometimes 25 to 50 times too large. The tool still surfaces them; the outline needs real editing.

What I set out to learn

“Learn to set up a 3D image processing pipeline end to end — and prove I can build something that holds up on real-world data.”

Week 1 takeaway

1 The pipeline exists

Indexing through training, evaluation, registry, endpoint, and interface

2 2D → 3D taught me what changes

Memory is the constraint; how you crop is a modelling decision, not preprocessing

3 Real data was the part I underestimated

Broken masks, wild geometry, four scanner makers — auditing WAS the work

How AI actually shaped the work

What I planned

A coding accelerator and an explainer.

What it became

An adversarial reviewer I stopped trusting by default.

- The loop: write a spec → a second AI reviews the plan → write code → review the code → a regression test for every bug found
- It found my leakage bug — and it had also written that bug
- The consistent failure mode: AI is confident about data it has not inspected
- So the rule became: make it read the file and show me the output before it asserts anything

If I'd had more time

These aren't regrets — they were deferred deliberately. I hit the goals I set for this project.

1

Remove the provided region

Let the system find the pancreas itself, so a raw scan goes in and a result comes out

2

A decision layer for specificity

The AUC says the signal is there — the fix is a gate in front of the model, not a timid model

3

Train on all 9,000 scans

Data scale was the only lever that consistently moved accuracy, and I never found its ceiling

Where it landed

96%

of tumors flagged for review

0.474

lesion Dice vs a ~0.53 benchmark

901

held-out scans, scored once

A working 3D pipeline, an honest evaluation, and a tool a radiologist could actually sit down with.

Questions